

A simple Compiler for a small arithmetic Script Language Mini Lab Project

Submitted By

Student Name	Student ID
Student-1 Name Md.Ebrahim Hossen Rana	Student-1 ID 232-15-292
Student-2 Name Md.Sadikur Rahman	Student-2 ID 232-15-852
Student-3 Name Mohammad Tanvir Hasan	Student-3 ID 232-15-127
Student-4 Name Sadiah Shuvra	Student-4 ID 0242310005101 370
Student-5 Name	Student-5 ID

MINI LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course **CSEXXX:**
Subject Name in the Computer Science and Engineering
Department



DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

December 9, 2025

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Name of the course teacher Mushfiquir Rahman , course teacher's Designation Assistant Professor** , Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

Course Teacher's Name:Mushfiquir Rahman

Designation:Assistant Professor

Department of Computer Science and Engineering

Daffodil International University

Submitted by

Student Name :Ebrahim Hossen Rana Student ID:232-15-292 Dept. of CSE, DIU	
Student Name :Md.Sadikur Rahman Student ID:232-15-852 Dept. of CSE, DIU	Student Name :Amar : Mohammad Tanvir Hasan ID:232-15-127 Dept. of CSE, DIU
Student Name :Sadia Shuvra Student ID:02423100051013 70	Student Name Student ID: Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:

Table 1: Course Outcome Statements

CO's	Statements
CO1	Understand the working procedure of a compiler for debugging programs
CO2	Analyze the role of syntax and semantics of Programming languages in compiler construction.
CO3	Apply the techniques, tools used in Compiler Construction in the design and construction of the phases of a compiler's components. algorithms, and different
CO4	Create a project by explaining complex computer engineering activities with the computer engineering community by performing effective communication through demonstration and presentation

Table 2: Mapping of CO, PO, Blooms, KP and CEP

CO	PO	Blooms	KP	CEP
CO1	PO1	C1, C2	KP1-4	EP1
CO2	PO2	C3, C4	KP1-4, KP6	EP1,EP3
CO3	PO3	C4, A2	KP6	EP1,EP3
CO4	PO10	C6, A2, P3	KP4	EP3,EP5

The mapping justification of this table is provided in section **4.3.1**, **4.3.2** and **4.3.3**.

Table of Contents

Declaration	i
Course & Program Outcome	ii
1 Introduction	1
1.1 Introduction	1
1.2 Motivation	1
1.3 Objectives	1
1.4 Feasibility Study	1
1.5 Gap Analysis	1
1.6 Project Outcome	1
2 Proposed Methodology/Architecture	2
2.1 Requirement Analysis & Design Specification	2
2.1.1 Overview	2
2.1.2 Proposed Methodology/ System Design	2
2.1.3 UI Design	2
2.2 Overall Project Plan	2
3 Implementation and Results	3
3.1 Implementation	3
3.2 Performance Analysis	3
3.3 Results and Discussion	3
4 Engineering Standards and Mapping	4
4.1 Impact on Society, Environment and Sustainability	4
4.1.1 Impact on Life	4
4.1.2 Impact on Society & Environment	4
4.1.3 Ethical Aspects	4
4.1.4 Sustainability Plan	4
4.2 Project Management and Team Work	4
4.3 Complex Engineering Problem	4
4.3.1 Mapping of Program Outcome	4
4.3.2 Complex Problem Solving	4
4.3.3 Engineering Activities	5
Table of Contents	Table of Contents
5 Conclusion	6
5.1 Summary	6
5.2 Limitation	6
5.3 Future Work	6

Chapter 1

Introduction

Every chapter should start with 1-2 sentences on the outline of the chapter.

1.1 Introduction

A compiler translates high-level programs into lower-level instructions. The **MathScript Compiler** developed in this project is a simplified compiler designed to process mathematical expressions such as:

```
A = (5 + 3) * 2;  
B = A / 4 + 7;
```

The compiler handles:

- Variable assignment
- Arithmetic operations
- Operator precedence
- Expression evaluation

The core compiler phases:

1. **Lexical Analysis** — Using Flex and **FlexLexer.h**
2. **Syntax Analysis** — Parser for expression grammar
3. **Semantic Evaluation** — Symbol table + evaluation

Tools used:

- Flex (lexer generator)
- Bison/Yacc (parser)
- C++ (backend)
- **FlexLexer.h** for interfacing the generated lexer

1.2 Motivation

Understanding compiler design strengthens:

- Deep understanding of programming languages
- Debugging skills
- Logical thinking
- Ability to build custom scripting languages

This project provides hands-on experience with real compiler tools like Flex.

1.3 Objectives

- 🎬 Develop a working MathScript mini-compiler
- 🎬 Design a Flex-based lexical analyzer
- 🎬 Implement grammar for arithmetic expressions
- 🎬 Build a C++ evaluation engine

1.4 Feasibility Study

The project is feasible because:

- Flex and Bison are widely used and well-documented
- Expression grammar is simple to implement
- Similar academic projects exist
- Required environment supports Lex/Yacc tools

1.5 Gap Analysis

Existing sample compilers mainly evaluate expressions without:

- User-defined variable persistence
- Clear grammar for mixed expressions
- Clean modular design for educational purposes

This project aims to fill these gaps by providing a structured, modular mini-compiler with readable implementation.

1.6 Project Outcome

The major outcomes include:

- A functioning expression-evaluation compiler
- Hands-on experience with Flex/Bison
- Understanding of compiler front-end and semantic phases
- Team experience in software engineering workflow

Chapter 2

Proposed Methodology/Architecture

This chapter describes the system architecture, methodology, and design diagrams

2.1 Requirement Analysis & Design Specification

2.1.1 Overview

The MathScript compiler takes an input script, analyzes tokens, parses expressions, computes results, and displays output.

2.1.2 Proposed Methodology/ System Design

Pipeline Diagram (Textual Substitute):

Input → Lexer (Flex) → Parser (Bison) → AST Generation → Evaluation → Output

Modules found inside your project files:

- flex rules → token identification
- bison grammar → precedence handling
- C++ symbol table → variable storage
- evaluation logic → computes expressions

2.1.3 UI Design

(The project is command-line based, so UI = CLI output.)

2.2 Overall Project Plan

- Week 1 → Study Flex & Bison
- Week 2 → Implement Lexer
- Week 3 → Implement Parser
- Week 4 → Write evaluation engine
- Week 5 → Testing and debugging

Chapter 3

Implementation and Results

This chapter outlines implementation details and results

3.1 Implementation

The project folder contains:

- FlexLexer.h
- Bison grammar files
- Flex lexer files
- C++ computation modules
- Supporting headers

Features implemented:

- Tokenization of identifiers, integers, operators
- Grammar for:
 - addition/subtraction
 - multiplication/division
 - parentheses
 - assignments
- Symbol table for variable tracking
- Printing computed values

3.2 Performance Analysis

- 🎬 Evaluates expressions instantly ($O(n)$ parsing)
- 🎬 Supports nested parentheses
- 🎬 Correct operator precedence (Bison precedence rules)
- 🎬 Minimal memory use due to simple symbol table

3.3 Results and Discussion

Example Input:

```
A = 5 + 3 * 2;  
B = A / 4 + 7;
```

Output:

```
A = 11  
B = 9.75
```

The compiler successfully handles arithmetic expression evaluation and variable dependencies

Chapter 4

Engineering Standards and Mapping

This chapter evaluates engineering, ethics, sustainability, and team-based outcomes.

4.1 Impact on Society, Environment and Sustainability

4.1.1 Impact on Life

Helps students understand programming language foundations and debugging techniques

4.1.2 Impact on Society & Environment

No environmental impact; encourages technological skill growth.

4.1.3 Ethical Aspects

The project maintains originality and adheres to academic integrity

4.1.4 Sustainability Plan

Source code is modular and can be expanded for:

- Function definitions
- Control statements
- Complex expressions

4.2 Project Management and Team Work

Budget Analysis

Resource	Cost (BDT)
Laptop / PC	Already available
Flex/Bison Tools	Free
Text Editor / IDE	Free
Internet	Minimal

Alternate Budget:

In a lab environment, all tools remain free.

Team Collaboration

- Task distribution for lexer, parser, evaluation engine
- Weekly meetings
- Git-based code versioning (if used)

4.3 Complex Engineering Problem

4.3.1 Mapping of Program Outcome

In this section, provide a mapping of the problem and provided solution with targeted Program Outcomes (PO's).

Table 4.1: Justification of Program Outcomes

PO's	Justification
PO1	Understanding of compiler rules and debugging
PO2	Analysis of lexical/syntax structures
PO3	Application of Flex/Bison to build compiler

4.3.2 Complex Problem Solving

In this section, provide a mapping with problem solving categories. For each mapping add subsections to put rationale (Use Table 4.2). For P1, you need to put another mapping with

Knowledge profile and rational thereof.

Table 4.2: Mapping with complex problem solving.

EP1 Depth of Knowledge	EP2 Range of Conflicting Requirements	EP3 Depth of Analysis	EP4 Familiarity of Issues	EP5 Extent of Applicable Codes	EP6 Extent of Stake- holder Involvement	EP7 Interdependence
✓	✓	✓	✓	✓	✓	✓

4.3.3 Engineering Activities

In this section, provide a mapping with engineering activities. For each mapping add subsections to put rationale (Use Table 4.3).

Table 4.3: Mapping with complex engineering activities.

EA1 Range of resources	EA2 Level of Interaction	EA3 Innovation	EA4 Consequences for society and environment	EA5 Familiarity
✓	✓	✓	✓	✓

Chapter 5

Conclusion

Every chapter should start with 1-2 sentences on the outline of the chapter.

5.1 Summary

This project successfully implemented a mini MathScript compiler capable of evaluating arithmetic expressions using Flex, Bison, and C++.

5.2 Limitation

- ❏ No support for floating-point precision control
- ❏ No function declarations
- ❏ No control structures (if, for, while)

5.3 Future Work

- Add variable type support
- Improve error handling
- Introduce loops & conditionals
- Build a GUI interface

References

- [1] Aho, Lam, Sethi, Ullman – Compilers: Principles, Techniques, Tools
- [2] Flex & Bison Documentation
- [3] Project files inside `MathScript Compiler.rar`